

Nova Corps Scramble

MIRAGE 2.0 — Binary Exploitation — Walkthrough

CATEGORY	DIFFICULTY	FILES
Binary Exploitation (Logic / Integer Overflow)	Medium	nova_corps_scramble
DEPLOYMENT	VULNERABILITY	STATUS
Offline binary — run locally	Unsigned 32-bit integer overflow	Solved

Step 1 — First Contact

The binary presents a small resource-management terminal: a Energy counter starting at 100, a Decoys counter starting at 0, and a three-option menu — Allocate Decoys, Recover Decoys, and Generate Master Override. Playing with it normally: allocating costs 10 Energy per decoy, recovering gives back 1 Energy per decoy. Starting from 100 Energy, there is no legitimate sequence of these two operations that increases your net Energy — it's a closed, lossy loop by design. That immediately raises the question the challenge name hints at (“Scramble”): the intended path is not to play by the stated rules, but to break them.

Step 2 — Recon and Reading the Arithmetic

The binary is not stripped, so `objdump -d -M intel` shows the three handlers directly. The “Generate Master Override” check reads:

```
cmp    dword [energy], 0xF423F    ; 0xF423F = 999,999
jbe    reject                    ; unsigned compare: Energy must be > 999999
```

That threshold (just over one million) is far beyond anything reachable by legitimately trading Energy and Decoys back and forth — confirming the “closed loop” observation from Step 1 wasn't a coincidence, it's the actual gate. Reading the allocate/recover handlers shows both `amount × 10` (the allocation cost) and every addition/subtraction are done in plain 32-bit registers, with no overflow check anywhere, and the input itself is parsed with `strtoul` straight into a 32-bit value — so any value the user types above $2^{32}-1$ silently wraps.

Step 3 — Turning That Into a Plan

Two properties combine here: the cost of allocating `amount` decoys is computed as $\text{amount} \times 10 \bmod 2^{32}$, and Decoys/Energy gained is `amount` itself, uncapped. Picking `amount = 2147483648` (`0x80000000`, i.e. 2^{31}) makes the cost $10 \times 2^{31} \bmod 2^{32} = 0$ — the allocation becomes free, while Decoys jumps to 2,147,483,648. Feeding that exact number back into “Recover Decoys” then adds the full 2,147,483,648 to Energy in one shot: $100 + 2147483648 = 2147483748$, comfortably past the one-million threshold — not earned through play, but produced by deliberately wrapping a 32-bit multiplication.

Step 4 — Running It

```
1
2147483648
2
2147483648
3
```

Sent in order to the binary's prompts: allocate 2,147,483,648 decoys at zero net cost, recover them for the same number of Energy, then request the Master Override with Energy now far above the threshold.

FLAG

```
MIRAGE{m4th_1s_h4rd_wh3n_b0unds_4r3_f1x3d}
```

Takeaway

Whenever a resource-trading loop looks intentionally lossy under normal play, it's worth asking what the underlying arithmetic is actually doing at the bit level — a “closed loop” in the game's stated rules can still be an open door if the implementation performs unchecked fixed-width arithmetic on values the player controls.